

SAPA Deployment & Integrasi (VPS Ubuntu + Nginx + Kubernetes + MQTT + AI Face Recognition)

Dokumen ini menjelaskan cara instalasi dan integrasi end-to-end SAPA di VPS Ubuntu Server dengan Nginx sebagai web server/reverse proxy, MongoDB berjalan di Kubernetes, dan kontrol gate via MQTT (ESP32). Termasuk juga opsi integrasi AI Face Recognition (di Edge atau di Server) untuk menghasilkan log presensi dan trigger pembukaan gate.

1) Gambaran Arsitektur

Komponen

- Frontend (Dashboard/Edge): React (Vite) berjalan di browser.
- Backend API: FastAPI (Python) menerima event presensi, menyimpan log, dan publish perintah ke MQTT.
- Database:
 - PostgreSQL/SQLite: data user & employee (SQLAlchemy).
 - MongoDB: log presensi & audit (Mongo).
- MQTT Broker: Mosquitto/EMQX untuk komunikasi Backend ? ESP32.
- ESP32 (Gate Controller): subscribe ke topik gate, mengendalikan relay/servo untuk membuka gate.
- Kubernetes: menjalankan MongoDB (dan bisa juga menjalankan MQTT + Postgres + Backend).
- Nginx di VPS: serve frontend (static) + reverse proxy ke Backend.

Alur Data (ringkas)

- 1) Edge melakukan capture wajah.
- 2) AI Face Recognition menentukan employee_id (di Edge atau di Backend/Service).
- 3) Edge kirim event ke Backend (HTTP) + opsional kirim frame untuk monitoring.
- 4) Backend:
 - tulis log presensi ke MongoDB
 - publish perintah gate ke MQTT ('sapa/gate')
- 5) ESP32 menerima perintah MQTT dan membuka gate.

Topik MQTT yang digunakan (sesuai backend saat ini)

- 'sapa/attendance' (ESP32/Edge ? Backend): event presensi (JSON).
- 'sapa/gate' (Backend ? ESP32): perintah gate (JSON).

2) Prasyarat

VPS

- Ubuntu Server 22.04/24.04 (direkomendasikan 22.04 LTS).
- Domain: misal 'sapa.example.com' mengarah ke IP VPS (A record).
- Akses SSH (key-based).

Software

- Nginx (di host VPS)
- Python 3.11 + venv (di host VPS untuk backend) atau container runtime (jika backend di K8s).
- Node.js 18+ (build frontend) atau build di CI.
- Kubernetes di VPS:
 - opsi 1: k3s (single node) di VPS (paling simple)
 - opsi 2: cluster K8s terpisah (butuh konektivitas jaringan antara VPS ? cluster)

3) Pilihan Deployment yang Direkomendasikan

Jika MongoDB harus berada di Kubernetes, opsi paling mudah dan ?valid? adalah:

Opsi A (Direkomendasikan): k3s di VPS + MongoDB/MQTT/Postgres di K8s + Nginx di host

- Kelebihan: konektivitas MongoDB/MQTT internal mudah, tanpa expose port sensitif ke internet.
- Kekurangan: perlu resource VPS cukup.

Opsi B: Cluster K8s terpisah + VPS Nginx + Backend di VPS

- Kelebihan: beban DB di cluster.
- Kekurangan: perlu desain jaringan (VPN/private link) agar VPS dapat akses MongoDB dengan aman.

Dokumen ini fokus ke Opsi A (paling aman dan praktis). Di bagian akhir ada catatan untuk Opsi B.

4) Instalasi Dasar VPS (Ubuntu)

- 1) Update OS dan paket:

- 'sudo apt update && sudo apt -y upgrade'
- 2) Buat user deploy dan SSH key:
 - 'sudo adduser deploy'
 - 'sudo usermod -aG sudo deploy'
 - Copy public key ke '~deploy/.ssh/authorized_keys'
- 3) Firewall dasar (UFW):
 - 'sudo ufw allow OpenSSH'
 - 'sudo ufw allow 80/tcp'
 - 'sudo ufw allow 443/tcp'
 - 'sudo ufw enable'
- 4) Set timezone:
 - 'sudo timedatectl set-timezone Asia/Jakarta'

5) Install Nginx + TLS (Let's Encrypt)

- 1) Install Nginx:
 - 'sudo apt install -y nginx'
- 2) Install Certbot:
 - 'sudo apt install -y certbot python3-certbot-nginx'
- 3) Buat konfigurasi Nginx (lihat contoh di bagian 9).
- 4) Aktifkan HTTPS:
 - 'sudo certbot --nginx -d sapa.farhn.dev'
- 5) Reload:
 - 'sudo nginx -t && sudo systemctl reload nginx'

6) Kubernetes di VPS: K8s ?penuh? vs k3s

Jawaban singkat: di VPS Ubuntu, kamu bisa ?pakai K8s? dengan 'kubeadm' (Kubernetes upstream), tapi untuk single-node VPS, yang paling praktis dan valid adalah **k3s** (karena k3s itu tetap Kubernetes, hanya lebih ringan dan mudah dioperasikan).

Kapan pakai k3s (direkomendasikan untuk kasus ini)

- Single node VPS.
- Butuh cepat jalan dan minim kompleksitas.
- MongoDB/Mosquitto berjalan di cluster yang sama dengan aplikasi.

Kapan pakai Kubernetes upstream (kubeadm)

- Kamu butuh cluster multi-node yang ketat (HA), dan tim sudah terbiasa operasional kubeadm.
- Ada kebutuhan plugin CNI/CSI tertentu yang spesifik.

Dokumen ini memakai **k3s** karena paling sesuai dengan target ?deploy di VPS Ubuntu + Nginx?.

7) Install Kubernetes (k3s) di VPS

- 1) Install k3s:
 - 'curl -sfL https://get.k3s.io | sh -s - --disable traefik'
- 2) Cek status:
 - 'sudo kubectl get nodes'
- 3) (Opsional) konfigurasi kubectl untuk user deploy:
 - 'sudo cat /etc/rancher/k3s/k3s.yaml | sed \"s/127.0.0.1/127.0.0.1/\" > /home/deploy/.kube/config'
 - 'sudo chown -R deploy:deploy /home/deploy/.kube'

8) Deploy MongoDB (dan Mosquitto/Postgres) di Kubernetes

Repo ini sudah punya manifest di folder 'k8s/'. Untuk production, tambahkan PVC. Minimal valid untuk jalan:

- 1) Apply namespace:
 - 'kubectl apply -f k8s/namespace.yaml'
- 2) Deploy MongoDB:
 - 'kubectl apply -f k8s/mongo-deployment.yaml'
- 3) Deploy MQTT (Mosquitto) di K8s:
 - 'kubectl apply -f k8s/mqtt-deployment.yaml'
- 4) (Opsional) Deploy Postgres di K8s:
 - 'kubectl apply -f k8s/db-deployment.yaml'
- 5) Cek service:
 - 'kubectl -n sapa get svc'

Catatan production:

- MongoDB dan Postgres sebaiknya pakai PVC + backup.
- MQTT sebaiknya pakai auth (username/password) dan TLS.

9) Deploy Backend di VPS (systemd) dan konek ke service di k3s

8.1 Install dependency backend

1) Install Python + build tools:

- 'sudo apt install -y python3.11 python3.11-venv python3-pip build-essential'

2) Clone repo:

- 'cd /opt && sudo git clone <REPO_URL> sapa'
- 'sudo chown -R deploy:deploy /opt/sapa'

3) Setup venv:

- 'cd /opt/sapa/backend'
- 'python3.11 -m venv venv'
- './venv/bin/pip install -r requirements.txt'

8.2 Konfigurasi environment backend

Buat file env: '/opt/sapa/backend/.env' (contoh):

- 'DATABASE_URL=postgresql://user:password@<POSTGRES_HOST>:5432/sapa_db'
- 'MONGODB_URI=mongodb://sapa-mongo.sapa.svc.cluster.local:27017'
- 'MONGODB_DB=sapa'
- 'MONGODB_COLLECTION_ATTENDANCE=attendance_logs'
- 'MONGODB_COLLECTION_AUDIT=audit_logs'
- 'MQTT_BROKER=sapa-mqtt.sapa.svc.cluster.local'
- 'MQTT_PORT=1883'
- 'SECRET_KEY=<isi-random-panjang>'
- 'EDGE_INGEST_KEY=<isi-random-panjang>' (opsional, jika ingin mengunci upload frame)

Jika Postgres tidak dipakai, bisa pakai SQLite:

- 'DATABASE_URL=sqlite:///opt/sapa/backend/attendance.db'

8.3 systemd service

Buat file '/etc/systemd/system/sapa-backend.service':

- WorkingDirectory: '/opt/sapa'
- ExecStart: './backend/venv/bin/python -m uvicorn backend.main:app --host 127.0.0.1 --port 8080'
- Load env dari '/opt/sapa/backend/.env'

Contoh isi '/etc/systemd/system/sapa-backend.service':

```
'''ini
[Unit]
Description=SAPA Backend (FastAPI)
After=network.target

[Service]
Type=simple
User=deploy
Group=deploy
WorkingDirectory=/opt/sapa
EnvironmentFile=/opt/sapa/backend/.env
ExecStart=/opt/sapa/backend/venv/bin/python -m uvicorn backend.main:app --host 127.0.0.1 --port 8080 --log-level info
Restart=always
RestartSec=2

[Install]
WantedBy=multi-user.target
'''
```

Lalu jalankan:

- 'sudo systemctl daemon-reload'
- 'sudo systemctl enable --now sapa-backend'
- 'sudo journalctl -u sapa-backend -f'

10) Deploy Frontend (build static) + Nginx reverse proxy

9.1 Build frontend

1) Install Node.js 18:

- 'curl -fsSL https://deb.nodesource.com/setup_18.x | sudo -E bash -'
 - 'sudo apt install -y nodejs'
- 2) Build:
- 'cd /opt/sapa/frontend'
 - 'npm ci'
 - 'npm run build'
- 3) Copy 'dist/' ke Nginx root:
- 'sudo mkdir -p /var/www/sapa'
 - 'sudo rsync -a --delete /opt/sapa/frontend/dist/ /var/www/sapa/'

10.2 Nginx site config

Buat '/etc/nginx/sites-available/sapa':

- server_name: 'sapa.farhn.dev'
- root: '/var/www/sapa'
- SPA fallback: 'try_files \$uri /index.html'
- reverse proxy API:
 - location '/api/' ? 'http://127.0.0.1:8080/'
 - pastikan rewrite menghapus prefix '/api'

Contoh konfigurasi '/etc/nginx/sites-available/sapa':

```
'''nginx
server {
    listen 80;
    server_name sapa.farhn.dev;

    root /var/www/sapa;
    index index.html;

    location / {
        try_files $uri /index.html;
    }

    location /api/ {
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;

        rewrite ^/api/(.*)$ /$1 break;
        proxy_pass http://127.0.0.1:8080;
    }
}'''
```

Aktifkan:

- 'sudo ln -s /etc/nginx/sites-available/sapa /etc/nginx/sites-enabled/sapa'
- 'sudo nginx -t && sudo systemctl reload nginx'

11) Integrasi AI Face Recognition

Ada 2 pendekatan yang ?valid?. Pilih satu:

Pendekatan 1 (Direkomendasikan): AI di Edge

- Edge device menjalankan proses AI (Python/ONNX) untuk:
 - face detection + face embedding
 - nearest neighbor match ke database embedding lokal (sinkron dari server)
- Output: 'employee_id', 'is_valid', dan metadata confidence.
- Edge mengirim hasil ke Backend:
 - 'POST /edge/events' dengan payload '{ is_valid, employee_id, message }'
 - 'POST /edge/frame' opsional (untuk monitor live di dashboard)

Keuntungan:

- Latensi rendah, lebih tahan jika internet lambat.
- Privasi lebih baik (tidak selalu kirim foto full ke server).

Pendekatan 2: AI di Server (Backend/Service)

- Edge hanya kirim foto (face crop) atau embedding ke backend.
- Backend memanggil service AI (microservice) untuk match.
- Backend menyimpan hasil dan publish MQTT.

Keuntungan:

- Edge lebih ringan.
- Model & data embedding terpusat.

Catatan implementasi:

- Simpan foto wajah employee saat registrasi (sudah ada upload face).
- Tambahkan pipeline ekstraksi embedding saat upload face, simpan embedding (di Postgres/Mongo).
- Saat presensi: ambil embedding baru dan lakukan matching.

12) Integrasi MQTT untuk Gate (ESP32)

11.1 Broker MQTT

Jika broker di k3s:

- Service 'sapa-mqtt' dapat diakses dari dalam cluster melalui DNS.
- Untuk ESP32 (di jaringan luar), gunakan salah satu:
 - expose MQTT via NodePort + firewall hanya untuk jaringan lokal
 - gunakan reverse proxy TCP (Nginx stream) + TLS
 - gunakan VPN (WireGuard) agar ESP32 masuk jaringan private (paling aman)

11.1.1 ESP32 satu jaringan (LAN) vs beda jaringan (Internet)

Mode A (sekarang): ESP32 satu jaringan yang sama dengan VPS

Target: ESP32 connect ke broker Mosquitto yang berjalan di k3s, tanpa membuka broker ke internet.

Opsi yang paling sederhana:

- Expose Mosquitto via NodePort, lalu batasi akses via firewall agar hanya LAN yang boleh masuk.

Langkah konsep:

- 1) Ubah service Mosquitto menjadi NodePort untuk port 1883 (dan 8883 jika pakai TLS).
- 2) Buka firewall hanya dari subnet LAN (mis. '192.168.0.0/24') ke port NodePort.
- 3) Konfigurasi ESP32:
 - MQTT host: IP VPS (LAN)
 - MQTT port: NodePort yang dipilih
 - username/password (wajib untuk production)

Catatan: NodePort biasanya berada di range 30000-32767.

Mode B (nanti): ESP32 beda jaringan

Target: tetap aman tanpa expose MQTT plaintext ke publik.

Pilihan yang direkomendasikan (urut aman ? praktis):

- 1) WireGuard VPN
 - VPS sebagai WireGuard server
 - ESP32 biasanya tidak bisa WireGuard langsung, jadi pakai gateway (Raspberry Pi/mini router) yang join VPN.
 - ESP32 connect MQTT ke gateway (LAN lokal), gateway meneruskan ke broker lewat VPN.
- 2) MQTT over TLS (port 8883) + firewall allowlist
 - Expose broker TLS saja
 - Tambahkan username/password + ACL
 - Allowlist IP publik jika memungkinkan (fixed IP). Jika IP dinamis, ini sulit.
- 3) Reverse proxy TCP (Nginx stream) + TLS termination
 - Nginx di host sebagai TCP proxy untuk MQTT TLS
 - Tetap perlu auth+ACL di broker

11.2 Format pesan (direkomendasikan)

Topik 'sapa/gate' (Backend ? ESP32):

```
'''json
{ "action": "open", "employee_id": "123456", "ts": "2026-05-06T13:00:00Z" }
'''
```

atau:

```
'''json
{ "action": "invalid", "reason": "unknown_face", "ts": "2026-05-06T13:00:00Z" }
'''
```

Topik 'sapa/attendance' (ESP32/Edge ? Backend):

```
'''json
{ "employee_id": "123456", "is_valid": true, "direction": "in" }
'''
```

ESP32 subscribe ke 'sapa/gate', lalu:

- jika 'action=open' ? aktifkan relay/servo selama N detik
- jika 'action=invalid' ? bunyikan buzzer/LED merah

13) Keamanan Minimal yang Wajib

- Jalankan web via HTTPS (wajib untuk akses camera dari browser Edge).
- Jangan expose MongoDB ke internet.
- Set 'EDGE_INGEST_KEY' dan kirim header 'X-EDGE-KEY' dari Edge/Dashboard publisher.
- Pakai auth MQTT (username/password) minimal, idealnya MQTT over TLS.
- Simpan 'SECRET_KEY' unik, panjang, dan rahasiakan.

13.1 Mosquitto (Auth + ACL) ? template konfigurasi

Untuk production, Mosquitto sebaiknya:

- 'allow_anonymous false'
- pakai password file
- pakai ACL untuk membatasi publish/subscribe

Contoh ACL konsep:

- user 'backend':
 - publish: 'sapa/gate'
 - subscribe: 'sapa/attendance'
- user 'esp32':
 - subscribe: 'sapa/gate'
 - publish: 'sapa/attendance'

Di repo ini sudah ditambahkan manifest siap pakai:

- 'k8s/mosquitto-production.yaml' (ConfigMap + PVC + Deployment + Service ClusterIP)
- 'k8s/mosquitto-nodeport.yaml' (Service NodePort untuk akses LAN)

Langkah apply (contoh):

- 1) Buat secret password file Mosquitto (format file 'passwords' hasil 'mosquitto_passwd'):
 - Buat file lokal 'passwords' berisi hash user 'backend' dan 'esp32'.
 - 'kubectl -n sapa create secret generic sapa-mosquitto-auth --from-file=passwords=./passwords'
- 2) Apply Mosquitto:
 - 'kubectl apply -f k8s/mosquitto-production.yaml'
- 3) Jika butuh akses dari ESP32 via LAN:
 - 'kubectl apply -f k8s/mosquitto-nodeport.yaml'
 - Buka UFW hanya dari subnet LAN ke port 31883.

Contoh UFW allowlist LAN (ganti subnet sesuai jaringan):

- 'sudo ufw allow from 192.168.0.0/24 to any port 31883 proto tcp'

14) Checklist Validasi (end-to-end)

- 1) Dashboard bisa diakses: 'https://sapa.example.com'
- 2) API health:
 - 'curl -i https://sapa.example.com/api/edge/status'
- 3) Edge publisher:
 - buka '/edge' atau aktifkan camera di dashboard
 - pastikan 'POST /api/edge/frame' sukses (200)
- 4) MongoDB:
 - pastikan log presensi masuk ke collection 'attendance_logs'
- 5) MQTT:
 - backend dapat connect ke broker
 - ESP32 menerima pesan 'sapa/gate' dan membuka gate

15) Catatan untuk Opsi B (Cluster K8s terpisah)

Jika MongoDB ada di cluster terpisah:

- Jangan gunakan NodePort publik untuk 27017.
- Gunakan koneksi private:

- VPN (WireGuard) antara VPS ? cluster network
- atau jalankan backend di cluster yang sama agar akses MongoDB tetap internal

Lampiran Production Hardening (Direkomendasikan untuk Deploy VPS)

Bagian ini melengkapi langkah ?jalan dulu? menjadi setup production yang lebih aman dan stabil: storage persisten, secrets, TLS, proteksi API, keamanan MQTT, backup, observability, dan rencana operasional.

A) Hardening VPS (Host Ubuntu)

A.1 Paket wajib

- 'sudo apt update && sudo apt -y upgrade'
- 'sudo apt install -y nginx ufw fail2ban ca-certificates curl git'

A.2 Firewall (UFW)

- Buka hanya port yang dibutuhkan:
 - 'sudo ufw allow OpenSSH'
 - 'sudo ufw allow 80/tcp'
 - 'sudo ufw allow 443/tcp'
- Jika kamu expose NodePort/SSH custom, sesuaikan.
- 'sudo ufw enable'

A.3 Fail2ban (minimal)

- Aktifkan jail untuk sshd (default biasanya sudah ada).
- Pastikan SSH pakai key-based dan disable password login untuk production.

A.4 Nginx security headers + rate limit

Tambahkan header keamanan di server block:

- 'X-Content-Type-Options: nosniff'
- 'X-Frame-Options: SAMEORIGIN'
- 'Referrer-Policy: no-referrer-when-downgrade'
- 'Content-Security-Policy' sesuai kebutuhan (hati-hati jika terlalu ketat).

Tambahkan rate limit untuk endpoint sensitif (login):

- limit '/api/login' untuk menahan brute force.

B) Hardening Kubernetes (k3s) di VPS

B.1 Install k3s dengan konfigurasi yang ?production-friendly?

Contoh (disesuaikan):

- disable Traefik (karena kamu pakai Nginx di host atau ingress-nginx):
 - 'curl -sL https://get.k3s.io | sh -s - --disable traefik'

B.2 Storage persisten (PVC)

Untuk single-node VPS, opsi paling sederhana:

- 'local-path-provisioner' bawaan k3s bisa dipakai untuk PVC.

Untuk production lebih serius:

- gunakan Longhorn (lebih robust) atau storage managed.

C) MongoDB Production (StatefulSet + PVC + Auth)

Manifest 'k8s/mongo-deployment.yaml' saat ini belum memakai PVC dan auth. Untuk production:

C.1 Gunakan Secret untuk kredensial

Buat secret:

- 'kubectl -n sapa create secret generic mongo-auth
- from-literal=MONGO_INITDB_ROOT_USERNAME=sapa
- from-literal=MONGO_INITDB_ROOT_PASSWORD='<password-kuat>''

C.2 StatefulSet + PVC (contoh minimal)

Gunakan StatefulSet agar nama pod stabil dan disk persisten. Contoh konsep (sesuaikan storageClass):

- Service: 'sapa-mongo' (ClusterIP)
- StatefulSet: volumeClaimTemplates untuk '/data/db'

C.3 Connection string backend

Gunakan auth:

```
-  
'MONGODB_URI=mongodb://sapa:<password>@sapa-mongo.sapa.svc.cluster.local:27017/?authSource=admin'
```

D) MQTT Production (Auth + ACL + TLS)

MQTT tanpa auth/TLS tidak aman untuk production.

D.1 Pilih broker

- Mosquitto: simple, ringan.
- EMQX: fitur enterprise-grade (ACL, dashboard, TLS lebih mudah).

D.2 Mosquitto (contoh hardening)

Gunakan ConfigMap + Secret:

- Password file: 'mosquitto_passwd'
- ACL file untuk membatasi topik:
 - ESP32 hanya subscribe 'sapa/gate' dan publish 'sapa/attendance' (jika diperlukan).
 - Backend publish 'sapa/gate' dan subscribe 'sapa/attendance' (jika backend juga subscribe).

D.3 TLS untuk MQTT

Opsi:

- MQTT TLS di port 8883 (langsung di broker).
- Jika ESP32 tidak support TLS dengan baik, gunakan WireGuard supaya traffic tetap private.

E) Edge: HTTPS + Device Identity

E.1 HTTPS wajib untuk camera web

Browser hanya izinkan 'getUserMedia()' di:

- 'https://domain' atau
- 'http://localhost'

Jika Edge device membuka dashboard lewat IP (http) di jaringan lokal, kamera bisa gagal.

Solusi:

- gunakan domain + HTTPS, atau
- jalankan Edge sebagai aplikasi lokal (native / python) atau PWA dengan HTTPS lokal.

E.2 Identitas Edge & autentikasi ingest

Untuk endpoint ingest frame:

- set 'EDGE_INGEST_KEY' di backend
- kirim header 'X-EDGE-KEY' dari Edge/Dashboard publisher

Untuk lebih aman:

- gunakan token per device (mis. 'EDGE_DEVICE_ID' + secret)
- simpan device registry di DB dan lakukan rotate token.

F) Backend Production (systemd hardening + health + log)

F.1 systemd hardening (opsional)

Tambahkan:

- 'NoNewPrivileges=true'
- 'PrivateTmp=true'
- 'ProtectSystem=strict'
- 'ProtectHome=true'
- 'ReadWritePaths=/opt/sapa/backend/uploads /opt/sapa/backend/attendance.db'

F.2 Reverse proxy (Nginx) yang benar

Pastikan Nginx meneruskan header dan tidak cache untuk frame:

- Endpoint '/api/edge/frame.jpg' sudah set 'Cache-Control: no-store'.

F.3 Environment via file

Jangan hardcode secret di unit file atau repo. Simpan di '/opt/sapa/backend/.env' dengan permission ketat:

- 'sudo chown deploy:deploy /opt/sapa/backend/.env'
- 'sudo chmod 600 /opt/sapa/backend/.env'

G) Database SQL (Postgres) Production

Jika kamu pakai Postgres:

- jalankan Postgres juga di K8s atau managed DB.
- gunakan PVC + backup.
- simpan 'DATABASE_URL' dengan password kuat.

H) Ingress/TLS untuk Kubernetes (opsional)

Karena kamu pakai Nginx di host untuk web, kamu tidak harus expose Ingress K8s untuk publik. Tetap bisa:

- hanya expose service internal cluster
- backend di VPS akses via DNS '*.svc.cluster.local'

Jika kamu ingin semua via ingress-nginx:

- pasang 'ingress-nginx' di k3s
- pasang 'cert-manager' untuk TLS otomatis
- map host 'sapa.example.com' ke ingress

I) WireGuard (Direkomendasikan untuk ESP32/MQTT)

Jika ESP32 berada di jaringan berbeda, cara paling aman:

- setup WireGuard server di VPS
- ESP32 (atau gateway lokal) join VPN
- MQTT broker tidak perlu public exposure

Jika ESP32 tidak bisa WireGuard:

- gunakan gateway (Raspberry Pi) yang join WireGuard dan jadi ?bridge? untuk ESP32.

J) Backup & Restore

J.1 MongoDB backup

Gunakan 'mongodump' terjadwal (CronJob di K8s) dan simpan ke:

- object storage (S3 compatible) atau
- volume terpisah (bukan disk yang sama).

J.2 Postgres backup

Gunakan 'pg_dump' terjadwal.

J.3 Disaster checklist

- cadangkan file '.env'
- catat versi image/tag yang digunakan
- simpan manifest K8s yang dipakai

K) Observability

Minimal yang disarankan:

- Nginx access/error log
- 'journalctl -u sapa-backend'
- dashboard metrics (sudah ada endpoint system metrics)

Jika butuh lebih:

- Prometheus + Grafana
- Loki untuk log

L) CI/CD (opsional tapi direkomendasikan)

L.1 Build container image

- backend image: 'your-registry/sapa-backend:<tag>'
- frontend image atau static build artifact (copy to '/var/www/sapa')

L.2 Deploy

- GitHub Actions / GitLab CI:
 - build + push image
 - apply manifest k8s
 - reload systemd/nginx jika host-based

M) AI Face Recognition Production Checklist

M.1 Data wajah (enrollment) yang aman

- simpan foto wajah employee di server untuk audit (opsional)

- simpan embedding (vector) untuk matching
- hindari menyimpan raw frame presensi tanpa kebutuhan

M.2 Matching strategy

- threshold cosine distance
- multi-sample per employee
- liveness detection (opsional) untuk anti spoofing

M.3 Latency & reliability

- jika AI di Edge: pastikan model ter-cache dan bisa jalan offline
- jika AI di server: pastikan queue/retry dan timeouts